INFORME DE LABORATORIO

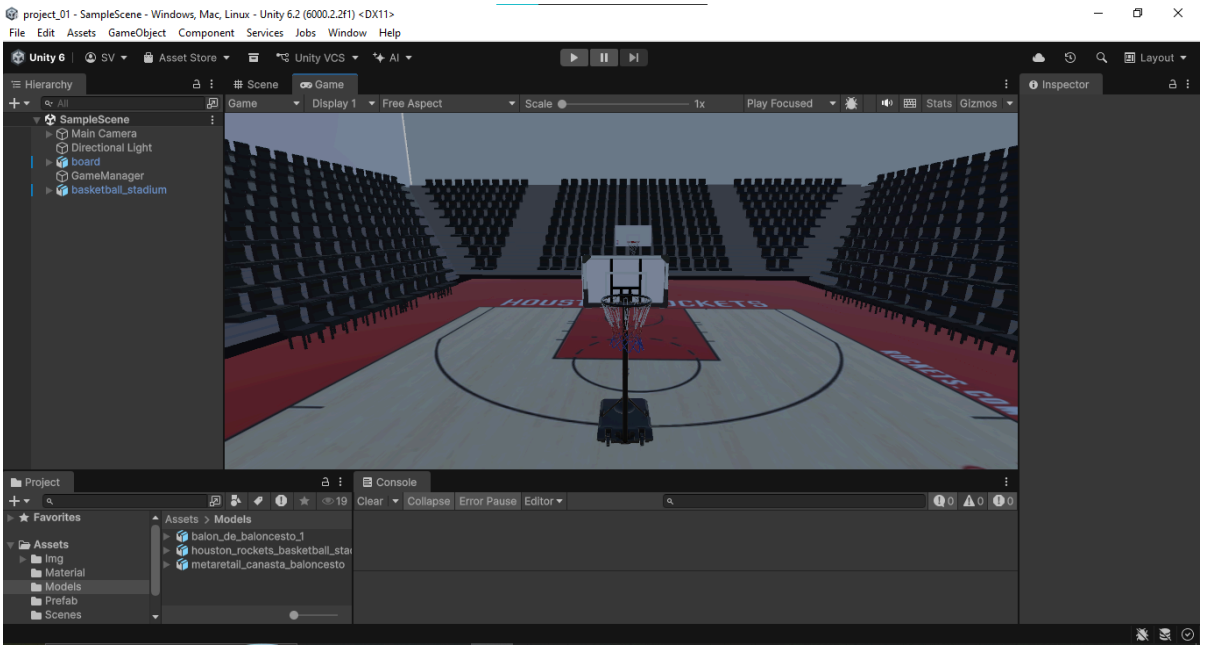
INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación Gráfica, Visión Computacional y Multimedia				
TÍTULO DE LA PRÁCTICA:	*Laboratorio 1*				
NÚMERO DE PRÁCTICA:	*01*	**AÑO LECTIVO:**	*2026-A*	**NRO. SEMESTRE:**	*IX (noveno)*
FECHA DE PRESENTACIÓN:	*22/04/2026*	**HORA DE PRESENTACIÓN:**	*12:20*		
INTEGRANTE (s): - Riveros Valeriano Sebastian Alfredo				**NOTA:**	
DOCENTE(s): *M. Sc. Ing. Rene Alonso Nieto Valencia*					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

1. Introducción

El presente laboratorio consiste en el desarrollo de una aplicación multimedia interactiva utilizando el motor de física de Unity. El proyecto implementa un sistema de lanzamiento de proyectiles en un entorno tipo baloncesto, donde el usuario puede controlar la dirección y fuerza del disparo.



2. Objetivos

2.1 Objetivo General

Desarrollar una aplicación interactiva que utilice un motor de física para simular el lanzamiento de proyectiles y la detección de colisiones.

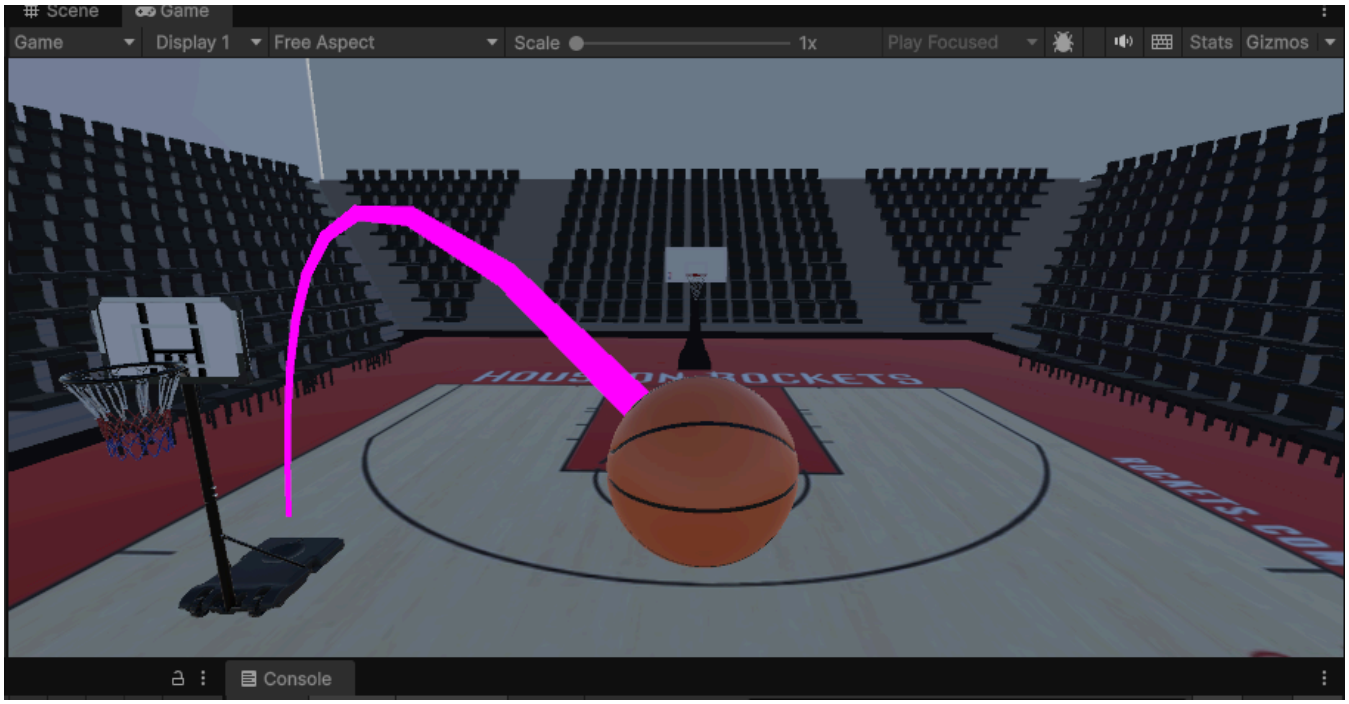
2.2 Objetivos Específicos

- Implementar el lanzamiento de proyectiles utilizando propiedades físicas como velocidad y gravedad.
- Aplicar detección de colisiones para validar eventos dentro del entorno.
- Diseñar un sistema de puntuación basado en la interacción del usuario.

3. Descripción de la Aplicación

La aplicación desarrollada es un juego de tipo baloncesto en el cual el usuario lanza una pelota con el objetivo de encestarla en un aro.

La mecánica principal consiste en que el jugador apunta con el mouse hacia una posición en el entorno y realiza el lanzamiento. La fuerza del disparo depende de la distancia entre la pelota y el punto seleccionado, generando así diferentes trayectorias.



El entorno está compuesto por los siguientes elementos:

- Una pelota controlada mediante física.
- Un aro de baloncesto con zonas de detección (superior e inferior).
- Un tablero que se mueve dinámicamente para aumentar la dificultad.

El sistema evalúa si la pelota atraviesa correctamente el aro para otorgar puntos al jugador.

4. Arquitectura del Sistema

```
/mnt/c/Users/sxrv/UNSA/semestre-2026_A/4-CGVCM_Lab_B/lab_01/project_01/Assets/Scripts 09:50:41
❯ tree
.
├── balon_cont.cs
├── balon_cont.cs.meta
├── board_move.cs
├── board_move.cs.meta
├── downTrigger.cs
├── downTrigger.cs.meta
├── game_manager.cs
├── game_manager.cs.meta
├── hoop_score.cs
├── hoop_score.cs.meta
├── spawner.cs
├── spawner.cs.meta
├── upTrigger.cs
└── upTrigger.cs.meta

1 directory, 14 files
```

El sistema está organizado en múltiples componentes que trabajan de manera conjunta para lograr la funcionalidad del juego.

- **BallController**: Controla el lanzamiento de la pelota, calcula la velocidad inicial y muestra la trayectoria antes del disparo.

```
    void DrawTrajectory()
    {
        line.positionCount = points;

        Vector3 startPos = transform.position;

        Vector3 target = GetMousePointOnGround();
        Vector3 dir = target - startPos;

        float distance = Mathf.Clamp(dir.magnitude, 0, maxForce);
        Vector3 direction = dir.normalized;

        Vector3 velocity = new Vector3(
            direction.x,
            heightFactor,
            direction.z
        ) * distance;

        for (int i = 0; i < points; i++)
        {
            float t = i * timeStep;

            Vector3 pos = startPos
                        + velocity * t
                        + 0.5f * Physics.gravity * t * t;

            line.SetPosition(i, pos);
        }
    }
```

- **BallSpawner**: Se encarga de generar nuevas pelotas en la escena.

```
    public void SpawnBall()
    {
        if (!GameManager.instance.IsGameActive()) return;

        GameObject ball = Instantiate(ballPrefab, spawnPoint.position, spawnPoint.rotation);
```

```
        BallController bc = ball.GetComponent<BallController>();
        bc.Activate(this);
    }
```

- **HoopScore**: Gestiona la lógica de validación de las canastas.

```
    public void ExitBottom(GameObject ball)
    {
        // BLOQUEO TOTAL
        if (scoredBalls.Contains(ball)) return;

        if (validBalls.Contains(ball))
        {
            validBalls.Remove(ball);

            // marcar ANTES de sumar
            scoredBalls.Add(ball);

            GameManager.instance.AddScore();
        }
    }
```

- **Triggers (upTrigger y downTrigger)**: Detectan la entrada y salida de la pelota en el aro.

- **GameManager**: Controla el puntaje y el tiempo de juego.

```
    [Header("Puntaje")]
    public int score = 0;
    public int pointsPerScore = 2;

    [Header("Tiempo de juego")]
    public float gameTime = 60f;
    private float currentTime;

    private bool isGameActive = true;
```

- **BackboardMovement**: Introduce movimiento al tablero para incrementar la dificultad.

Flujo del sistema:

El usuario apunta → se calcula la trayectoria → se lanza la pelota → se detectan colisiones → se valida la canasta → se actualiza el puntaje.

5. Implementación

5.1 Lanzamiento físico

El lanzamiento se implementa mediante el uso de un componente Rigidbody. Inicialmente, la pelota se mantiene en estado cinemático sin gravedad. Al momento del disparo, se activa la física y se asigna una velocidad inicial calculada en función de la dirección y distancia hacia el punto objetivo.

```
    void Shoot()
    {
        isActive = false;
        line.positionCount = 0;

        rb.isKinematic = false;
        rb.useGravity = true;

        Vector3 target = GetMousePointOnGround();
```

```
        Vector3 dir = target - transform.position;

        float distance = Mathf.Clamp(dir.magnitude, 0, maxForce);
        Vector3 direction = dir.normalized;

        Vector3 velocity = new Vector3(
            direction.x,
            heightFactor,
            direction.z
        ) * distance;

        rb.linearVelocity = velocity;

        StartCoroutine(Respawn());
    }
```

5.2 Trayectoria

Se utiliza un LineRenderer para visualizar la trayectoria del proyectil antes del lanzamiento. Esta se calcula utilizando una aproximación de la ecuación del movimiento parabólico, considerando la velocidad inicial y la gravedad.

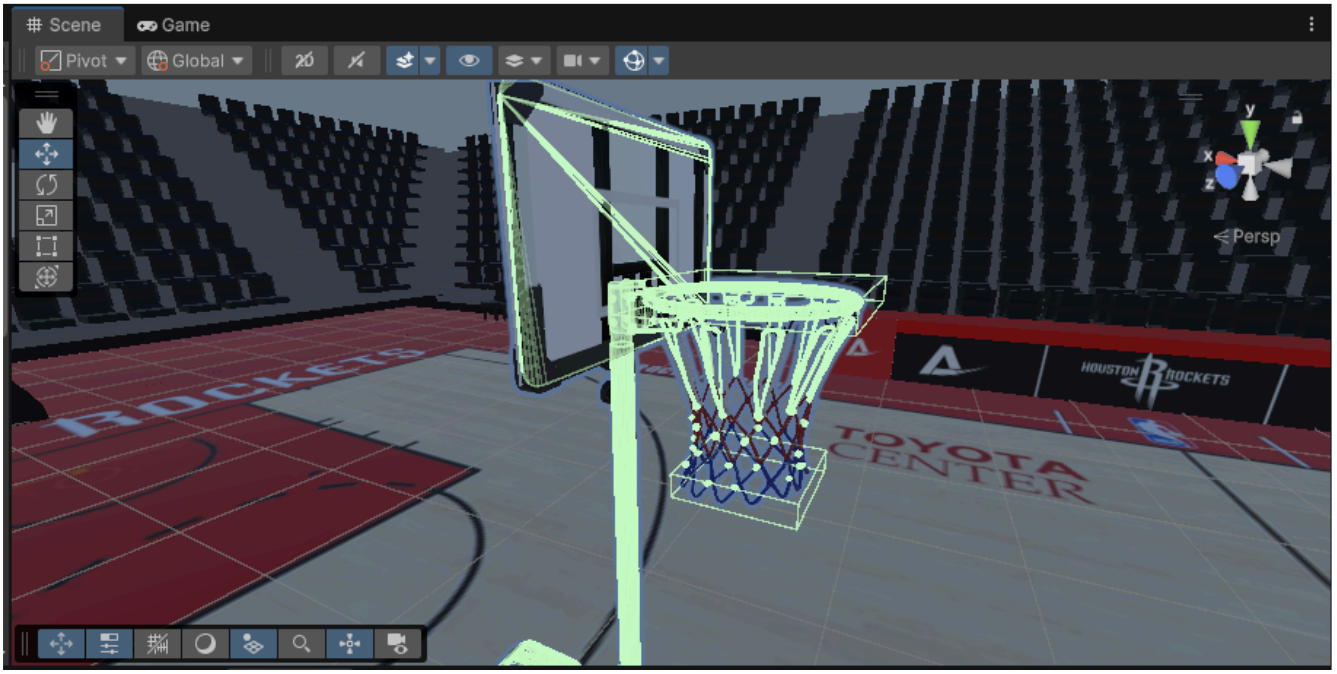
```
    void DrawTrajectory()
    {
        line.positionCount = points;

        Vector3 startPos = transform.position;

        Vector3 target = GetMousePointOnGround();
        Vector3 dir = target - startPos;

        float distance = Mathf.Clamp(dir.magnitude, 0, maxForce);
        Vector3 direction = dir.normalized;

        Vector3 velocity = new Vector3(
            direction.x,
            heightFactor,
            direction.z
        ) * distance;

        for (int i = 0; i < points; i++)
        {
            float t = i * timeStep;

            Vector3 pos = startPos
                        + velocity * t
                        + 0.5f * Physics.gravity * t * t;

            line.SetPosition(i, pos);
        }
    }
```

5.3 Colisiones

El sistema de colisiones se basa en triggers ubicados en la parte superior e inferior del aro.

- Si la pelota entra por la parte superior, se registra como válida.
- Si posteriormente sale por la parte inferior, se confirma la canasta y se asignan puntos.

Esto evita errores como contar puntos cuando la pelota atraviesa el aro de manera incorrecta.

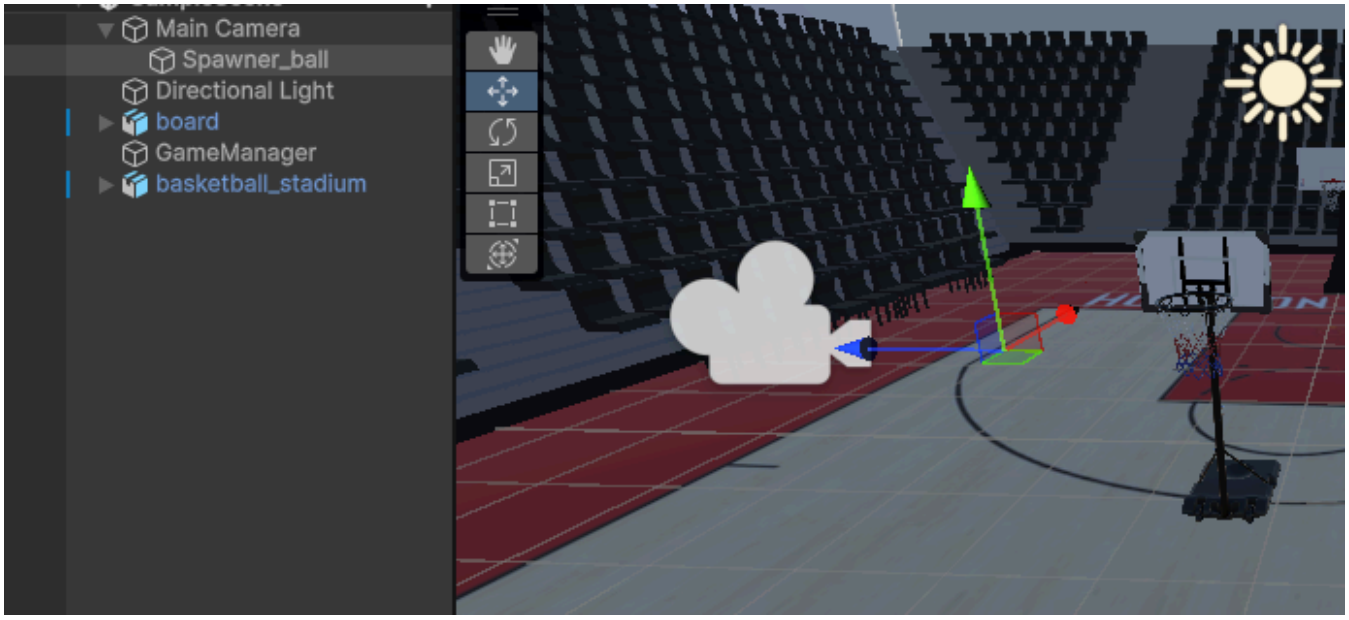


5.4 Sistema de juego

El GameManager controla el estado del juego, incluyendo el tiempo límite y el puntaje acumulado. Cuando el tiempo finaliza, el juego se detiene automáticamente.

5.4 Spawner

Un Empty Object Ubicado en frente de la camara para controlar la ubicacion de el spawn de los balones.

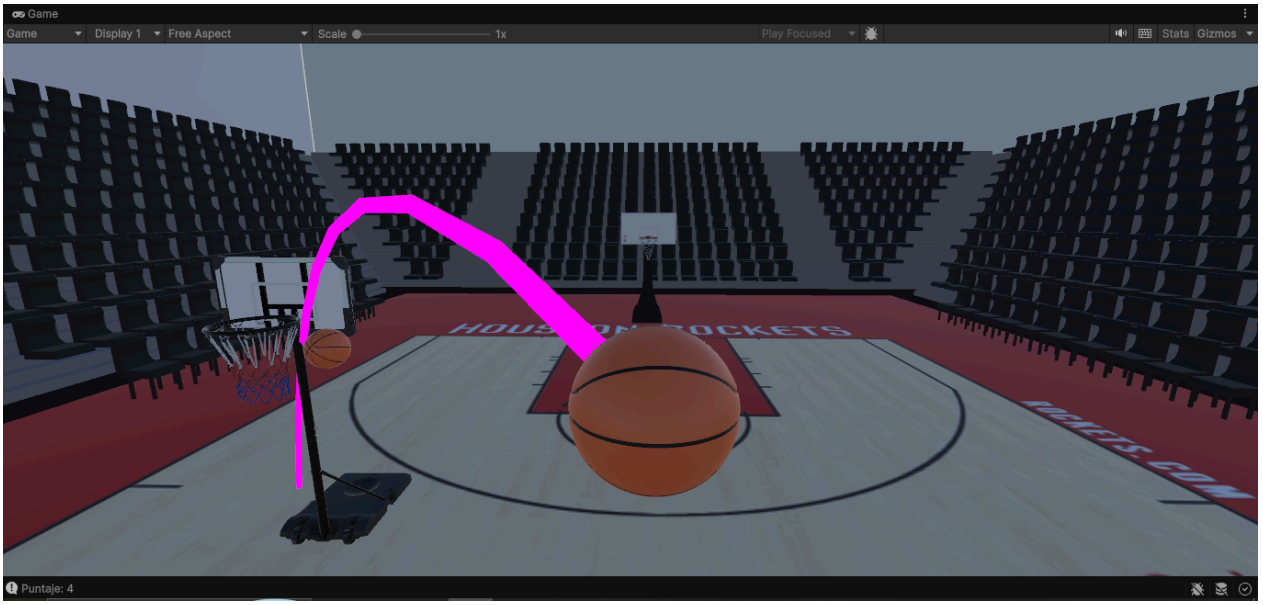


6. Resultados

La aplicación desarrollada permite simular de manera efectiva el lanzamiento de proyectiles utilizando física realista. La trayectoria responde correctamente a la fuerza aplicada y a la gravedad, generando un comportamiento coherente.

El sistema de colisiones funciona de manera precisa, validando únicamente las canastas realizadas correctamente. Asimismo, el puntaje se actualiza en función de las acciones del usuario.

La inclusión de elementos dinámicos, como el movimiento del tablero, incrementa la dificultad y mejora la experiencia de interacción.



II. ANEXOS

Carpeta con Scripts del proyecto:

https://drive.google.com/drive/folders/1ZMM6wZ3HoqhWzVkw2IaHhHqs1R718ZPc?usp=sharing

Video demostrativo:

https://drive.google.com/file/d/12vTrRpKQbcC6APFD0-S7xYaSZ329iGeY/view?usp=sharing

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] Gordon, V. S., & Clevenger, J. L. (2020). Computer Graphics Programming in OpenGL with C++. Stylus Publishing, LLC.

[2] D. P. Kothari, G. Awari, D. Shrimankar , A. Bhende (2019), Mathematics for Computer Graphics and Game Programming: A Self-Teaching Introduction